

TradeLog — Characterization & Summary

Course project · Track 3 (Software Product – AI App for Investors) · BGU · 2026 Team: Shahar Navian – 325895332 · Ben Rubinovitz – 325706463 · Idan Halilov – 325350643 **Live app:** <https://tradelog-peach.vercel.app> · **Repository:** github.com/Benru1503/tradelog · **Research notebook:** [ml/tradelog_prediction.ipynb](#)

1. The Financial Problem

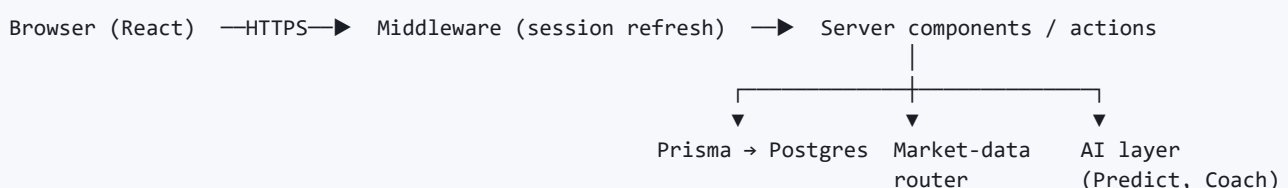
Retail investors trading stocks, crypto, and forex face three compounding problems TradeLog addresses, two of them directly with AI:

1. **Trading platforms don't surface enough statistics.** A bank's brokerage app, a crypto exchange, or a forex broker gives a balance and a raw trade history — not the statistics that actually explain performance (win rate, hold-time patterns, cash-adjusted returns). It's difficult for a user to genuinely follow their own account.
2. **No forward-looking signal.** A self-directed investor has no accessible tool that estimates the probability a position moves up or down over the next day/week, stated with an honest confidence level rather than a black-box "buy/sell" call.
3. **No behavioral feedback, in a form people already trust.** LLMs are now part of daily life — people ask them almost everything — yet trading behavior gets no such assistant. Outcomes are dominated by habits (holding losers too long, oversizing after a loss, ignoring one's own edge), but nothing surfaces these patterns or comments on them the way an experienced trader would, useful to new and veteran investors alike.

TradeLog's two AI features map onto problems 2 and 3: **Predict** (a trained direction-forecasting model with a stated confidence) and **Coach** (an LLM turning a trader's history into behavioral findings, that experienced voice). Problem 1 is solved by the product itself — positions, cash flows, time-/money-weighted returns — which both AI features build on.

2. Product & Technological Architecture

TradeLog is a single Next.js 14 (App Router, TypeScript) application backed by Supabase (Postgres + Google OAuth, EU region) and deployed on Vercel in the same Frankfurt region as its database — no separate backend service. Pages are React Server Components. Every mutation runs through a server action or API route, so the browser never talks to the database or holds a provider API key.



Core surfaces: trade log with positions and averaged cost, cash-flow ledger powering TWR/MWR returns, analytics (equity curve, sector heatmap, allocation), a watchlist, and a sandboxed "Playground" for what-if/DCA simulation. Market data (Finnhub for quotes, Yahoo Finance for historical candles, CoinGecko for crypto) is server-side only and degrades to a `–` placeholder rather than crashing.

Engineering & security. Every query is user-scoped, RLS is enabled on every table, and the public/anon database role holds no privileges — a deliberate lockdown after an audit found tables reachable through the public API key. Provider and model keys live only in server processes, verified by scanning the deployed bundle. Sign-in is restricted to Google in application code, not a dashboard setting, since the auth API is reachable directly with the public key. Market-data and AI paths are rate-limited per user. CI gates every push on lint, type-check, formatting, a build, and **148 unit tests** (incl. golden-vector Python↔TypeScript parity tests, §3.1). **11 Playwright e2e specs** run against a real database outside CI.

The two AI models

	Predict	Coach
Task	Next-day / next-week price direction, with confidence	Behavioural review of a trader's own history
Technology	XGBoost (gradient-boosted trees), trained offline	Gemini (LLM), called at request time
Input	19 scale-free features from daily closes + volume	Precomputed statistics + recent journal-note excerpts
Output	P(up), direction, confidence	Structured findings (JSON, schema-validated)
Serves	Any ticker, price history fetched without an API key	The signed-in user's own trade history

3. The AI Models — Technical Detail

3.1 Predict — direction forecasting with confidence

Trained on 14 liquid assets (BTC, ETH, 10 large-cap stocks, SPY, QQQ), daily closes 2020→present, split strictly chronologically: train up to 2024-12, validation 2025-01→2025-09, held-out **test window** from 2025-10 the model never saw during training, early stopping, or threshold selection. Two XGBoost classifiers — **d1** (next bar) and **w1** (next 5 bars) — train on 19 scale-free features (log returns, SMA ratios, Cutler's RSI, rolling volatility, distance from the 20-bar range, a volume z-score, calendar encoding, an asset-class flag). No alternative data (on-chain, macro, sentiment) ships in production — it must score any ticker from keyless price history at request time.

Training happens in Python, inference happens in TypeScript, inside the Next.js server, and the two must agree exactly — so every feature is implemented with plain loops and a finite lookback

(≤ 50 bars) rather than library indicators or recursive smoothing whose value depends on how much history was fetched. The trainer exports a plain-JSON tree dump plus a **measured** intercept. A TypeScript evaluator walks the same trees with `float32` split comparisons, matching XGBoost's predictor bit-for-bit. Golden-vector tests pin features to 9 decimals and probabilities to 8 against the real model, so a formula changed on only one side fails the suite immediately.

Confidence, precisely defined: the model outputs `p(up)`, the class probability for "closes higher after the horizon." Direction is `UP` if `p(up)  $\geq$  0.5` else `DOWN`. The displayed confidence is `max(p(up), 1-p(up))`, i.e. how far the model's own probability sits from a coin flip. This is the model's raw, uncalibrated probability — a genuine, honest confidence signal, but not verified against a calibration study (see §5).

Each prediction is persisted per user and later resolved against a live price into a HIT/MISS outcome, so the app's own historical track record is visible in the UI rather than only backtested numbers.

3.2 Coach — behavioural analysis via LLM

Coach answers a different question: not "what will the market do," but "what does this trader's own history say about how they trade." Every number the model can cite — win rate, hold-time asymmetry, payoff ratio, revenge-trade rate, sizing discipline, per-tag/per-weekday performance — is computed deterministically in TypeScript from the user's own rows *before* the model sees the data. The LLM (Gemini, `gemini-flash-latest`) receives that fact sheet and is instructed only to interpret it. Output is constrained to a fixed JSON schema and re-validated with Zod on the way back, so a hallucinated response fails loudly rather than rendering next to real P&L. The exact fact sheet behind every report is stored and shown in the UI, so any claim is auditable.

This targets the standard failure mode of LLM-generated financial content — a model asked to "analyse these trades" will confidently invent an average holding time. One handed `avgHoldHoursLosers: 12.4` can only repeat it. A report requires ≥ 5 closed trades, is cached by a hash of the fact sheet (unchanged history reuses the stored report), and each user is capped at 10 generations/24h, since one shared API key serves the whole install.

4. Results

4.1 Predict — production model (test window 2025-10 → 2026-07, 2,900–2,956 held-out rows)

Horizon	Test AUC	Test accuracy	Base rate (always "up")
d1 (next day)	0.531	51.7%	51.8%
w1 (next week)	0.527	51.7%	51.6%

Accuracy sits at or slightly below the naïve baseline. AUC modestly above 0.5 indicates a faint *ranking* signal rather than reliable directional accuracy — the honest state of daily-horizon

forecasting, and stated as such in the product UI itself.

Backtest (long when $p(\text{up}) \geq 0.55$, flat otherwise, 10 bps fee per position change), production model:

Asset	Strategy	Buy & hold	Days in market	Max drawdown	Sharpe
BTC	+9.7%	-46.1%	29%	-14.9%	0.54
AAPL	-5.8%	+30.5%	27%	-15.3%	-0.40
SPY	-0.1%	+12.3%	54%	-6.6%	0.04

[Chart: [charts/backtest-comparison.html](#) — strategy vs. buy & hold, BTC/AAPL/SPY]

The BTC result looks strong, and the drawdown column shows why: it's drawdown-avoidance — the model sat out most of a 46% fall — not directional skill. On the two assets that rallied, stepping out cost money: a regime-dependent effect, not a general edge.

The research notebook ([ml/tradelog_prediction.ipynb](#)) independently compares three models on the identical leak-free split — XGB-lite (deployed features only), XGB-full (+ on-chain and macro data), and an LSTM:

Model	Test AUC	Accuracy	Base rate
XGB-lite	0.512	51.8%	51.8%
XGB-full	0.494	51.6%	51.8%
LSTM	0.500	51.3%	51.1%

[Chart: [charts/model-comparison.html](#) — test AUC vs. coin flip, XGB-lite/XGB-full/LSTM]

All three sit at the edge of noise. Alternative data did not help. **How fragile the backtest is.** The notebook applies the same backtest rules to its own alternative-data model (XGB-full) on the same price window, and BTC comes out at **-7.0%** instead of +9.7%, while buy-and-hold is identical in both runs (-46.1%) — proving the entire gap is the choice of model. Fee sensitivity on that run: -1.7% at 0 bps, -7.0% at 10 bps, -14.3% at 25 bps.

[Chart: [charts/fee-sensitivity.html](#) — BTC (XGB-full) return at 0/10/25 bps]

So the honest answer to the course's alpha question is the notebook's: **no consistent positive alpha net of fees.** The +9.7% is one draw from a distribution over model variants and cost assumptions. The one defensible effect is drawdown avoidance in a falling market.

4.2 Coach — what it actually surfaces

Coach's result is judged by whether it turns a trader's history into findings a person can use. In testing it reliably picked up things like a hold-time ratio showing losers held several times longer than winners, or a cluster of revenge trades right after a loss with a below-baseline win rate — patterns a trader rarely notices about their own habits, stated plainly instead of as generic advice like "cut your losses earlier."

What makes those findings trustworthy, not just plausible-sounding, is architectural: every number Coach cites is computed deterministically before the model sees the data, and its JSON output is schema-validated on every response, so it can interpret numbers but never invent or recompute them. That layer is fully unit-tested end to end.

5. Limitations & Risks (Caveats)

Predict:

- **Single test window** (9 months, one regime). Walk-forward evaluation across multiple windows is the right next step before trusting the edge further.
- **Threshold (0.55) chosen a priori**, not optimised. Probabilities are **uncalibrated** — the confidence number is honest but unverified against a reliability curve.
- **Model-choice sensitivity**: swapping the feature set for the notebook's alternative-data variant flips the sign of BTC's backtest on identical prices.
- **Serving-time domain shift**: crypto trains on Yahoo `BTC-USD` but serves from CoinGecko. Venue closes differ by tens of basis points.
- **Calendar-based scoring, bar-based label**: a prediction resolves 24h (D1) or 7d (W1) after it was made, an approximation of the trained bar horizon. An exactly-flat outcome counts as a MISS.
- **Basket bias**: all 14 assets are large, liquid names that survived to 2026 — a mild survivorship tilt.
- Depends on Yahoo Finance's unofficial keyless endpoint. On failure the UI shows "historical data unavailable" rather than crashing.

Coach:

- Runs on Google AI Studio's **free tier** — quotas are per-account and shared by all users (hence the 10-reports-per-day cap), and the free tier permits using submitted content to improve Google's products. The privacy page discloses this — the opt-out is simply not using the feature.
 - The live network call is exercised manually, not covered by the automated suite (the deterministic fact layer is fully unit-tested).
 - Free-text journal notes flow into the prompt. The system instruction constrains the model to treat them strictly as data, output is schema-constrained with no tool use — but prompt injection is a design consideration, not a mathematically eliminated risk.
-

6. Future Work

- **Walk-forward retraining** on a schedule, replacing the single frozen 2026-07 snapshot.
- **Probability calibration study** (reliability curves) so displayed confidence is verified, not just honestly computed.
- **Keyless macro features** (SPX/VIX/DXY) at serving time — cheap to add without breaking the "no API key" constraint.
- **Position-aware alerts** — notify a user when the model's call flips on a ticker they hold.
- **A document-grounded assistant** (RAG over filings) alongside Predict and Coach.